OSSP-SKILL-WEEK3

Session5:

Task1: To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Source code:

```
  GNU nano 7.2
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_QUOTES 20

int main() {
    char input[MAX_INPUT];
    char quoted_strings[MAX_QUOTES][MAX_INPUT];
    int quote_count = 0;

    printf("Session 5 - Single Quote Parsing\n");
    printf("Enter a command with single quotes:\n");
    printf("myshell> ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    int i = 0;

    while (input[i] != '\0') {

        if (input[i] == '\'') {
            i++;

            int j = 0;

            while (input[i] != '\0' && input[i] != '\'') {
                if (j < MAX_INPUT - 1) {
                    quoted_strings[quote_count][j++] = input[i];
                }
                i++;
            }

            quoted_strings[quote_count][j] = '\0';
```

```
            quoted_strings[quote_count][j] = '\0';

            if (input[i] == '\'') {
                quote_count++;
                i++;
            } else {
                printf("Error: Unclosed single quote.\n");
                return 1;
            }
        } else {
            i++;
        }
    }

    printf("\nQuoted strings found:\n");

    if (quote_count == 0) {
        printf("No single-quoted strings found.\n");
    } else {
        for (int k = 0; k < quote_count; k++) {
            printf("Quoted String %d: [%s]\n",
                   k + 1, quoted_strings[k]);
        }
    }

    printf("\nParsing validation:\n");

    if (quote_count > 0)
        printf("Single-quote parsing successful.\n");
    else
        printf("No quoted content to validate.\n");

    return 0;
}
```

Output:

```
root@LAPTOP-1COJKKG6:~# nano session5_task1.c
root@LAPTOP-1COJKKG6:~# gcc session5_task1.c -o session5_task1
root@LAPTOP-1COJKKG6:~# ./session5_task1
Session 5 - Single Quote Parsing
Enter a command with single quotes:
myshell> echo 'Hello $USER'

Quoted strings found:
Quoted String 1: [Hello $USER]

Parsing validation:
Single-quote parsing successful.
root@LAPTOP-1COJKKG6:~#
```

Observation: I learned how to process single-quoted strings and preserve their contents as literal text. I understood that variable expansion should not occur inside single quotes. I also learned how to store quoted strings, validate the parsing result, and handle errors such as an unclosed single quote.

Task2: To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Source code:


GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX_INPUT 300
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 100
void expand_variable(char *input, char *output) {
int i = 0;
int j = 0;
while (input[i] != '\0' && j < MAX_INPUT - 1) {
if (input[i] == '$') {
i++;
char variable[50];
int k = 0;
while (isalnum((unsigned char)input[i]) || input[i] == '_') {
if (k < 49)
variable[k++] = input[i];
i++;
}
variable[k] = '\0';
char *value = getenv(variable);
if (value != NULL) {
int v = 0;
while (value[v] != '\0' && j < MAX_INPUT - 1) {
output[j++] = value[v++];
}
}
} else {
output[j++] = input[i++];
}
}
output[j] = '\0';
}
int main() {
char input[MAX_INPUT];
char expanded[MAX_INPUT];
char tokens[MAX_TOKENS][MAX_TOKEN_LENGTH];
^G Help
^O Write Out
^W Where Is
^K Cut
^T Execu
^X Exit
^R Read File
^\ Replace
^U Paste
^J Justi


GNU nano 7.2
int token_count = 0;
int i = 0;
printf("Session 5 - Double Quote Parsing\n");
printf("Double quotes preserve spaces and allow variable expansion.\n");
printf("Enter a command:\n");
printf("myshell> ");
fgets(input, sizeof(input), stdin);
input[strcspn(input, "\n")] = '\0';
if (strlen(input) == 0) {
printf("Empty command.\n");
return 0;
}
/* Parse input */
while (input[i] != '\0' && token_count < MAX_TOKENS) {
while (isspace((unsigned char)input[i])) {
i++;
}
if (input[i] == '\0')
break;
int j = 0;
/* Double-quoted string */
if (input[i] == '"') {
i++;
while (input[i] != '\0' && input[i] != '"') {
if (j < MAX_TOKEN_LENGTH - 1)
tokens[token_count][j++] = input[i];
i++;
}
if (input[i] == '"') {
i++;
} else {
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```
GNU nano 7.2
            } else {
                printf("Error: Unclosed double quote.\n");
                return 1;
            }

            tokens[token_count][j] = '\0';
            token_count++;
        }

        /* Normal token */
        else {
            while (input[i] != '\0' &&
                   !isspace((unsigned char)input[i]) &&
                   input[i] != '"') {

                if (j < MAX_TOKEN_LENGTH - 1)
                    tokens[token_count][j++] = input[i];

                i++;
            }

            tokens[token_count][j] = '\0';

            if (j > 0)
                token_count++;
        }
    }

    printf("\nTokens before variable expansion:\n");

    for (int k = 0; k < token_count; k++) {
        printf("Token %d: [%s]\n", k + 1, tokens[k]);
    }

    printf("\nVariable expansion:\n");

    for (int k = 0; k < token_count; k++) {

        expand_variable(tokens[k], expanded);

        printf("Token %d: [%s]\n", k + 1, expanded);
    }
```

```
    printf("\nParsing validation:\n");
    printf("Double-quote parsing successful
    printf("Spaces inside quotes were prese
    printf("Variable expansion was processe

    return 0;
}
```

Output:

```
root@LAPTOP-1COJKKG6:~# gcc session5_task2.c -o session5_task2
root@LAPTOP-1COJKKG6:~# ./session5_task2
Session 5 - Double Quote Parsing
Double quotes preserve spaces and allow variable expansion.
Enter a command:
myshell> echo "Hello $USER"

Tokens before variable expansion:
Token 1: [echo]
Token 2: [Hello $USER]

Variable expansion:
Token 1: [echo]
Token 2: [Hello root]

Parsing validation:
Double-quote parsing successful.
Spaces inside quotes were preserved.
Variable expansion was processed.
root@LAPTOP-1COJKKG6:~#
```

Observation: I learned how to process double-quoted strings while preserving spaces within them. I understood how variable expansion can be performed inside double quotes, such as expanding environment variables like $USER. I also learned how to parse quoted and nested tokens, validate the parsing results, and detect errors such as unclosed double quotes.

Session6:

Task1: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

Source code:

```
  GNU nano 7.2
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 300

int main() {
    char input[MAX_INPUT];
    char output[MAX_INPUT];

    int i = 0;
    int j = 0;
    int escaped = 0;

    printf("Session 6 - Escape Sequence Parsing\n");
    printf("Enter a command with escaped spaces or special characters:\n");
    printf("myshell> ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty input.\n");
        return 0;
    }

    while (input[i] != '\0' && j < MAX_INPUT - 1) {

        if (escaped) {
            output[j++] = input[i];
            escaped = 0;
        }
        else if (input[i] == '\\') {
            escaped = 1;
        }
        else {
            output[j++] = input[i];
        }

        i++;
    }

    if (escaped) {
        printf("\nWarning: Escape character at the end of input.\n");
    }

    output[j] = '\0';

    printf("\nOriginal input:\n");
    printf("%s\n", input);

    printf("\nProcessed input:\n");
    printf("%s\n", output);

    printf("\nParser validation:\n");
    printf("Escape sequences processed successfully.\n");
    printf("Escaped characters were preserved.\n");

    return 0;
}
```

Output:

```
root@LAPTOP-1COJKKG6:~# gcc session6_task1.c -o session6_task1
root@LAPTOP-1COJKKG6:~# ./session6_task1
Session 6 - Escape Sequence Parsing
Enter a command with escaped spaces or special characters:
myshell> hello\ world

Original input:
hello\ world

Processed input:
hello world

Parser validation:
Escape sequences processed successfully.
Escaped characters were preserved.
root@LAPTOP-1COJKKG6:~#
```

Observation: I learned how to handle escape sequences while processing command input. I understood how escaped spaces and special symbols can be preserved as literal characters instead of being treated as separators or operators. I also learned how to detect escape characters, process escaped input, validate parser output, and handle complex user inputs.

Task2: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

Source code:

```
        if (WIFEXITED(status)) {
            printf("\nChild exited with status: %d\n",
                   WEXITSTATUS(status));
        } else {
            printf("\nChild terminated abnormally.\n");
        }

        printf("Parent process completed.\n");
    }

    return 0;
}

^G Help        ^O Write Out    ^W Where Is     ^K Cut
```

```
  GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    int status;

    printf("Session 6 - Process Execution\n");

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        /* Child process */
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Executing: ls -l\n");

        execlp("ls", "ls", "-l", NULL);

        /* This runs only if exec fails */
        perror("exec failed");
        exit(EXIT_FAILURE);
    }
    else {
        /* Parent process */
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent waiting for child...\n");

        waitpid(pid, &status, 0);

^G Help        ^O Write Out   ^W Where Is    ^K Cut
^X Exit        ^R Read File   ^\ Replace     ^U Paste
```

Output:

```
Parent PID: 536
Child PID: 537
Parent waiting for child...

Child Process
Child PID: 537
Parent PID: 536
Executing: ls -l
total 332
-rw-r--r-- 1 root root    53 Aug  8 06:04 destination.txt
drwxr-xr-x 5 root root  4096 Aug 11 06:52 ossp_2520090049
-rw-r--r-- 1 root root    41 Aug  8 08:32 sample.txt
-rwxr-xr-x 1 root root 16232 Aug 11 10:12 session2_task1
-rw-r--r-- 1 root root   559 Aug 11 10:15 session2_task1.c
-rwxr-xr-x 1 root root 16104 Aug 11 14:39 session2_task2
-rw-r--r-- 1 root root   537 Aug 11 14:39 session2_task2.c
-rwxr-xr-x 1 root root 16680 Aug 11 16:22 session3_task1
-rw-r--r-- 1 root root  4230 Aug 11 16:22 session3_task1.c
-rwxr-xr-x 1 root root 16400 Aug 11 16:34 session3_task2
-rw-r--r-- 1 root root  2188 Aug 11 16:34 session3_task2.c
-rwxr-xr-x 1 root root 16192 Aug 11 17:28 session4_task1
-rw-r--r-- 1 root root  2266 Aug 11 17:20 session4_task1.c
-rwxr-xr-x 1 root root 16280 Aug 11 17:41 session4_task2
-rw-r--r-- 1 root root  2252 Aug 11 17:40 session4_task2.c
-rwxr-xr-x 1 root root 16192 Aug 12 03:09 session5_task1
-rw-r--r-- 1 root root  1552 Aug 12 03:09 session5_task1.c
-rwxr-xr-x 1 root root 16320 Aug 12 11:44 session5_task2
-rw-r--r-- 1 root root  3236 Aug 12 11:53 session5_task2.c
-rwxr-xr-x 1 root root 16192 Aug 12 11:58 session6_task1
-rw-r--r-- 1 root root  1222 Aug 12 11:58 session6_task1.c
-rwxr-xr-x 1 root root 16360 Aug 12 12:02 session6_task2
-rw-r--r-- 1 root root  1190 Aug 12 12:02 session6_task2.c
-rwxr-xr-x 1 root root 16312 Aug 11 07:10 skill_week1_task2
-rw-r--r-- 1 root root   879 Aug 11 09:58 skill_week1_task2.c
-rw-r--r-- 1 root root    53 Aug  8 06:03 source.txt
-rwxr-xr-x 1 root root 16528 Aug  8 04:41 week1pt1
-rw-r--r-- 1 root root  1350 Aug  8 04:41 week1pt1.c
-rwxr-xr-x 1 root root 16224 Aug  8 06:03 week2pt1
-rw-r--r-- 1 root root  1208 Aug  8 06:02 week2pt1.c
-rwxr-xr-x 1 root root 16304 Aug  8 03:26 week3pt1
-rw-r--r-- 1 root root  1023 Aug  8 03:26 week3pt1.c
-rwxr-xr-x 1 root root 16136 Aug  8 03:33 week3pt2
-rw-r--r-- 1 root root   376 Aug  8 03:32 week3pt2.c

Child exited with status: 0
Parent process completed.
```

Observation: I learned how to create a child process using fork() and execute another program using exec(). I understood how to pass arguments to the executed program and how the parent process can wait for the child using waitpid(). I also learned how to handle execution errors and check the child's exit status using process-management functions.